

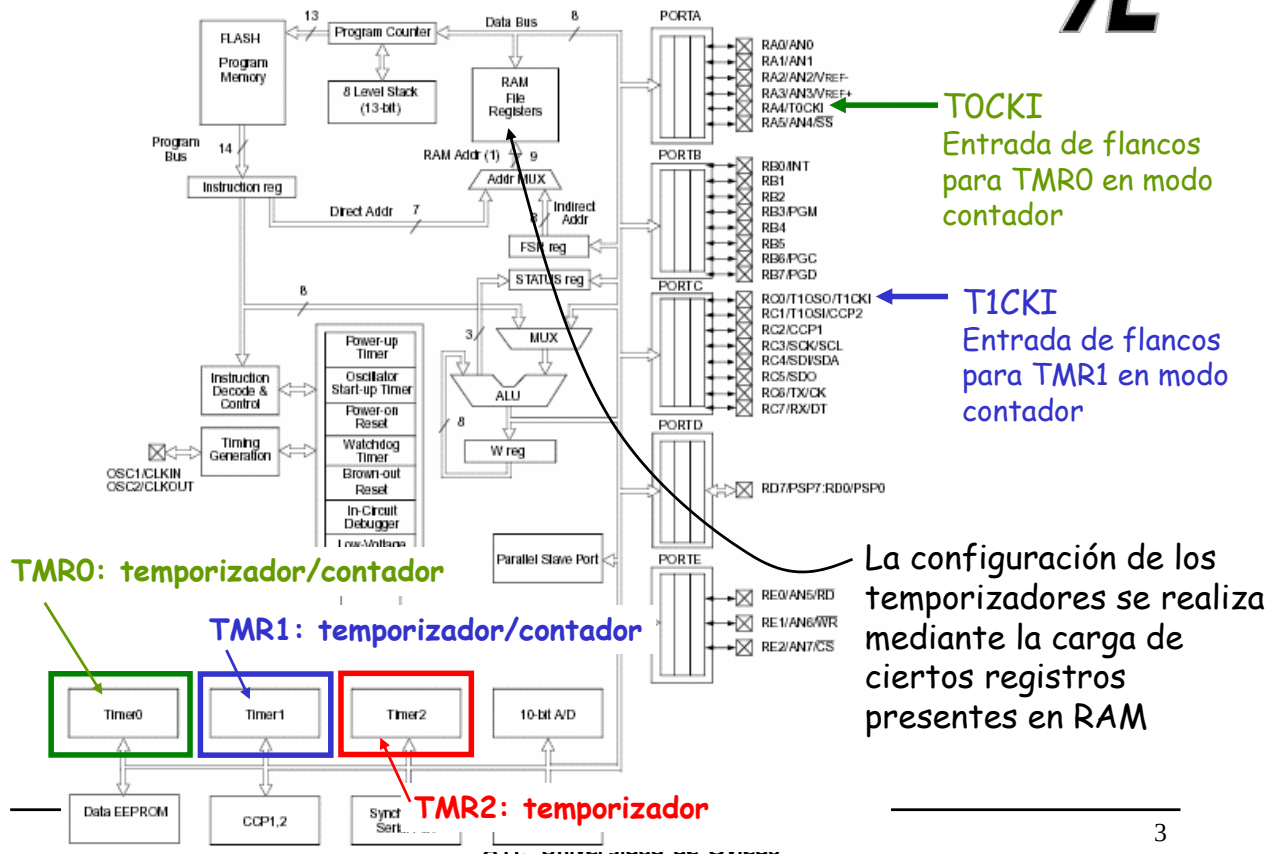


Módulos Temporizadores



CARACTERÍSTICAS GENERALES DE LOS TEMPORIZADORES

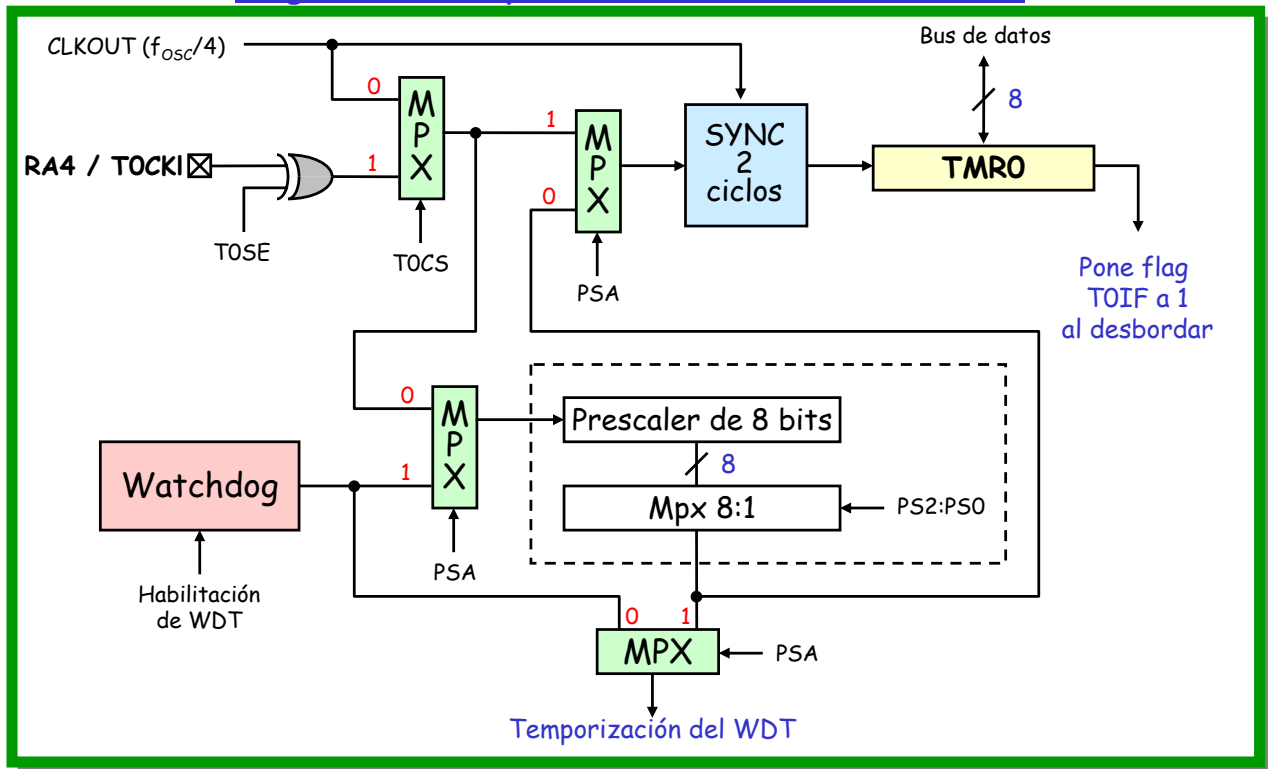
- Un temporizador, en general, es un dispositivo que marca o indica el transcurso de un tiempo determinado
- Los PIC16F87X tienen 3 módulos temporizadores denominados TIMER0 (TMR0), TIMER1 (TMR1) y TIMER2 (TMR2).
- Los módulos temporizadores en los microcontroladores PIC se emplean para contabilizar intervalos de tiempo o para contar flancos que aparecen en pines externos del micro, esto último lo pueden hacer TMR0 y TMR1 pero no TMR2
- Cuando trabajan como temporizadores, utilizan como patrón de cuenta un reloj que se genera a partir del oscilador del microcontrolador
- Cada módulo puede generar una interrupción para indicar que algún evento ha ocurrido (que se ha sobrepasado el valor máximo de cuenta de un temporizador -overflow- o que se ha alcanzado un valor dado)



Temporizador TMRO

- Se basa en un **contador ascendente de 8 bits** al que se accede mediante un registro en RAM denominado TMRO (posiciones 01h-101h).
- Dicho registro se puede **leer** (p.e. `movf TMRO,W`) y se puede **escribir** (`movwf TMRO`) **desde la CPU** del microcontrolador.
- Puede utilizar un **prescaler** o **divisor de frecuencia** previo de 8 bits cuyo valor de división es configurable por software.
- Se puede seleccionar como fuente de reloj: un **reloj interno** ($f_{osc}/4$) como temporizador o uno **externo** que entre a través del pin RA4/TOCKI como contador
- Permite **solicitar interrupciones** cuando se produce un **desbordamiento** (**overflow**) del registro TMRO. Es decir cuando pasa del valor 0xFF al 0x00.
- Para el caso de cuenta de pulsos de un reloj externo, se puede seleccionar en **qué flanco** (de subida o de bajada) se realiza la cuenta.

Diagrama de Bloques del TEMPORIZADOR TMRO



TEMPORIZADOR TMRO y WATCHDOG (WDT)

• Los bits de configuración que aparecen en el anterior diagrama de bloques están en el **registro OPTION** (denominado OPTION_REG en el fichero de inclusión de etiquetas de registros y bits P16F877.INC para distinguir el registro de la antigua instrucción OPTION)

• El **divisor de frecuencia** se le asigna bien al **TMRO** ó bien al **WDT** mediante el bit PSA.

• Si PSA=1, entonces el prescaler **es utilizado por el WDT y TMRO** contabiliza **directamente los flancos** sin división alguna

	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
	RBPV	INTEDG	TOCS	TOSE	PSA	PS2	PS1	PS0
bit 7								bit 0
bit 7	RBPV							
bit 6	INTEDG							
bit 5	TOCS: TMRO Clock Source Select bit							
	1 = Transition on TOCKI pin							
	0 = Internal instruction cycle clock (CLKO)							
bit 4	TOSE: TMRO Source Edge Select bit							
	1 = Increment on high-to-low transition on TOCKI pin							
	0 = Increment on low-to-high transition on TOCKI pin							
bit 3	PSA: Prescaler Assignment bit							
	1 = Prescaler is assigned to the WDT							
	0 = Prescaler is assigned to the Timer0 module							
bit 2-0	PS2:PS0: Prescaler Rate Select bits							
	Bit Value	TMRO Rate	WDT Rate					
	000	1:2	1:1					
	001	1:4	1:2					
	010	1:8	1:4					
	011	1:16	1:8					
	100	1:32	1:16					
	101	1:64	1:32					
	110	1:128	1:64					
	111	1:256	1:128					

TEMPORIZADOR TMR0

La **fente de reloj** para el TIMERO se selecciona mediante el **bit TOCS** (OPTION<5>).

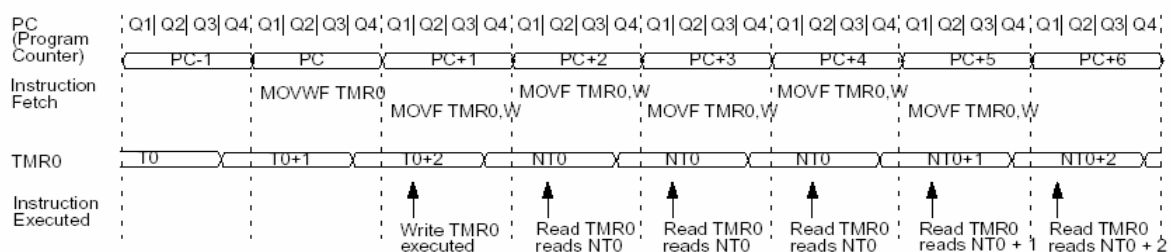
- Si TOCS=0, el TMR0 cuenta flancos a partir del reloj interno (frecuencia base $f_{osc}/4$).
- Si TOCS=1, el TMR0 cuenta como base flancos que entran al microcontrolador por el pin RA4/TOCKI.

Si se cuentan pulsos del pin RA4/TOCKI, el **bit TOSE** (OPTION<4>) permite seleccionar el **flanco de la señal** en el que se produce el incremento de la cuenta (entrada de la puerta EXOR)

- Si TOSE=0, se selecciona el flanco de subida.
- Si TOSE=1, se selecciona el flanco de bajada.

TEMPORIZADOR TMR0

Cuando se carga un valor en el registro TMR0 (se escribe mediante una instrucción), se produce un **retardo de dos ciclos de instrucción** durante los cuales se inhibe tanto el prescaler como TMR0. Será necesario tener en cuenta esa inhibición temporal a la hora de **realizar una precarga** (compensar sumando los ciclos de instrucción que "se pierden")



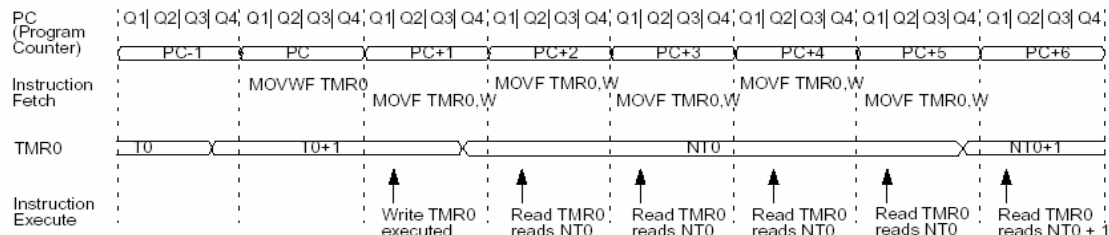
EJEMPLO DE CUENTA DE TMR0 SIN PRESCALER (PSA=1) Y FUENTE DE RELOJ INTERNA (TOCS=0)

Dado que cuando se realiza una escritura en el registro TMR0, el incremento del se inhibe durante los dos siguientes ciclos de instrucción.

TEMPORIZADOR TMR0

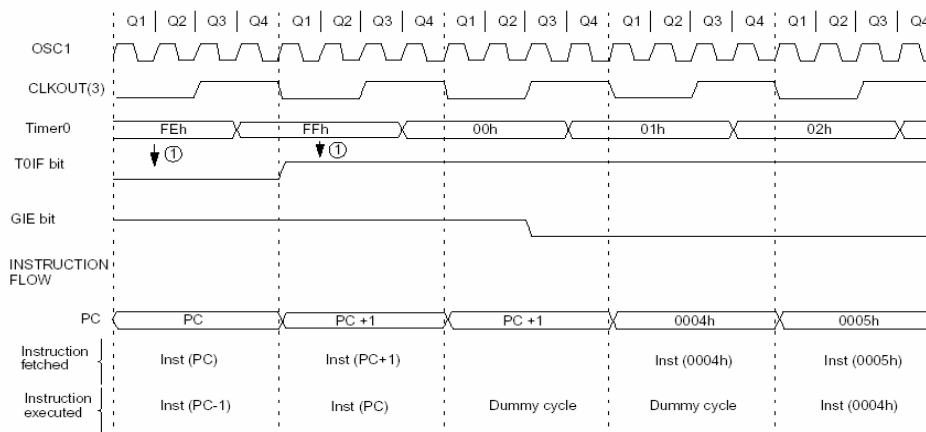
Si se utiliza la opción del prescaler (PSA=0), el TMR0 solo **se incrementa cada "n" flancos de reloj** (interno o externo). El valor del prescaler "n" viene definido por el valor de los bits PS2:PS0 (OPTION<2:0>) de acuerdo a la siguiente tabla:

Bit Value	TMR0 Rate	Bit Value	TMR0 Rate
000	1 : 2	100	1 : 32
001	1 : 4	101	1 : 64
010	1 : 8	110	1 : 128
011	1 : 16	111	1 : 256



EJEMPLO DE CUENTA DE TMR0 CON PRESCALER 1:2 (PSA=0 y PS<2:0>=000b) Y FUENTE DE RELOJ INTERNA (T0CS=0)
También a tener en cuenta que al realizar la escritura en TMR0, el incremento se inhibe durante los dos siguientes ciclos de instrucción.

TEMPORIZADOR TMR0 - INTERRUPTIONES



Note 1: Interrupt flag bit TOIF is sampled here (every Q1).
2: Interrupt latency = 4TCY where TCY = instruction cycle time.
3: CLKOUT is available only in RC oscillator mode.

La interrupción del TMR0 no puede despertar al microcontrolador del modo dormido, ya que el TIMERO está apagado durante el modo dormido, aún estando en modo contador, debido a la propia sincronización.

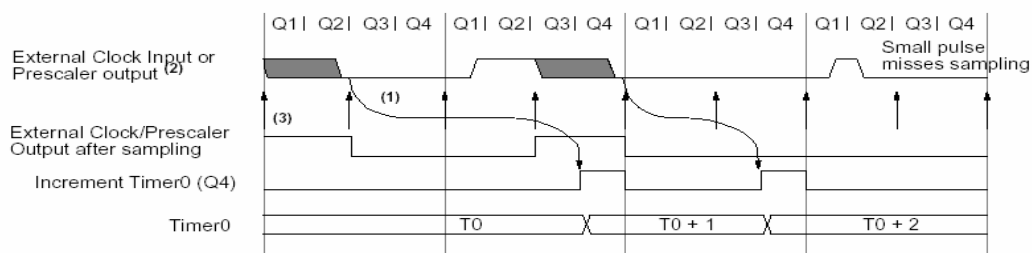
La interrupción del TMR0 se genera cuando en el registro **se produce un rebosamiento (overflow)** pasando del valor 0xFF a 0x00. Este "overflow" **pone a 1 el flag TOIF (INTCON<2>)**. Si el bit de **enmascaramiento particular TOIE (INTCON<5>)** y la **máscara global de interrupciones GIE (INTCON<7>)** están a "1", se produce el salto a la rutina de interrupción (posición 0x0004 de la memoria de programa). Antes de salir de la rutina de interrupción (RETFIE) del TMR0 **debe limpiarse el flag TOIF (BCF INTCON,TOIF por ejemplo)** ya que en caso contrario se produciría una nueva entrada en la misma.

TEMPORIZADOR TMRO - USO DEL RELOJ EXTERNO (SINCRONIZACIÓN)

Cuando se usa el reloj interno, el incremento del TMRO (si no se usa prescaler) o del prescaler si se usa, se realiza siempre **al finalizar el ciclo Q3** del ciclo de instrucción (ver figuras en transparencias anteriores).

Cuando se usa el **reloj externo**, se deben cumplir ciertos requisitos por parte de dicho reloj. Los requisitos aseguran que el reloj externo pueda sincronizarse con el reloj interno (Tosc). Todo ello lleva a que pueda existir un retardo entre el flanco en el pin RA4/T0CKI y el incremento real del TIMERO.

La **sincronización de la señal T0CKI** con el reloj interno se realiza por **muestreo de la señal de T0CKI** (si no se usa prescaler) ó de la salida del prescaler al final de los ciclos Q2 y Q4 de cada ciclo de instrucción



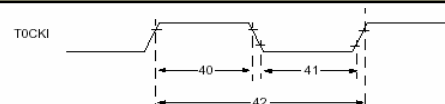
Note 1: Delay from clock input change to Timer0 increment is $3T_{osc}$ to $7T_{osc}$. (Duration of Q = T_{osc}).
Therefore, the error in measuring the interval between two edges on Timer0 input = $\pm 4T_{osc}$ max.
2: External clock if no prescaler selected, Prescaler output otherwise.
3: The arrows indicate the points in time where sampling occurs.

TEMPORIZADOR TMRO - USO DEL RELOJ EXTERNO (SINCRONIZACION)

Debido a este proceso de sincronización, la señal T0CKI, si no se usa prescaler, debe estar **a nivel alto al menos durante $2T_{osc}$ + un pequeño retardo RC de 20ns** y **a nivel bajo durante al menos otros $2T_{osc}$ + 20ns**.

Si se usa prescaler, la entrada de reloj externa se divide por el contador asíncrono que constituye el propio prescaler. Es decir, el **prescaler se incrementa en el momento en que se produce el flanco** (ó con un pequeño retardo de propagación propio). La señal que debe cumplir los requisitos de tener un periodo de $4T_{osc} + 40ns$ debe ser **la de salida del prescaler**. De esta forma, cuando se usa el el prescaler, los requisitos para la señal T0CKI es que el periodo sea de $4T_{osc} + 40ns$ dividido por el valor del prescaler. El único límite viene impuesto por un valor mínimo de 10 ns para el mínimo ancho de pulso.

Debido a este proceso de sincronización, va a existir un retardo (que puede estar entre $3T_{osc}$ y $7T_{osc}$) **desde que se produce el flanco en la señal T0CKI hasta que se produce el verdadero incremento de TMRO**. (Ver transparencia anterior lo que ocurre con los flancos de bajada)



Param No.	Symbol	Characteristic	Min	Typ	Max	Units	Conditions
40	T0H	T0CKI High Pulse Width	No Prescaler $0.5T_{CY} + 20$	—	—	ns	
		With Prescaler	10	—	—	ns	
41	T0L	T0CKI Low Pulse Width	No Prescaler $0.5T_{CY} + 20$	—	—	ns	
		With Prescaler	10	—	—	ns	
42	T0P	T0CKI Period	GREATER OF: $20 \mu s$ OR $T_{CY} + \frac{40}{N}$	—	—	ns	N = prescale value (1, 2, 4, ..., 256)

TEMPORIZADOR TMR0 - REGISTROS ASOCIADOS AL MODULO TMR0

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other resets
01h, 101h	TMR0	Timer0 module's register								xxxx xxxx	uuuu uuuu
0Bh,8Bh, 10Bh,18Bh	INTCON	GIE	PEIE ⁽¹⁾	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
81h, 181h	OPTION	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111
85h	TRISA	—	—	PORTA Data Direction Register ⁽¹⁾						--11 1111	--11 1111

El pin RA4/T0CKI **debe estar definido como entrada** si se utiliza como fuente de reloj para el TMR0, el reloj externo.

Todas las **instrucciones de escritura sobre el registro TMR0** (CLRF TMR0, MOVWF TMR0, BSF TMR0, bitx, etc.) **realizan una limpieza del prescaler.**

TEMPORIZADOR TMR0 - CAMBIO DEL PRESCALER

La asignación del prescaler al TMR0 (PSA=0) ó al WATCHDOG (PSA=1) puede realizarse **en cualquier momento del programa**. El cambio de la asignación del prescaler en medio de un programa, puede provocar un RESET no deseado en el microcontrolador debido **al desbordamiento del Watchdog** durante la ejecución del programa. Por ello, el cambio de la asignación del prescaler de un módulo a otro debe realizarse como se indica en los siguientes fragmentos de programas. Esta precaución debe tenerse en cuenta incluso si el WDT está deshabilitado.

;Cambio de prescaler (TIMER0 ->WDT)

```
BSF STATUS, RP0      ;Banco 1
MOVLW b'xx0xxxx'    ;Selección de fuente de reloj y valor
MOVWF OPTION_REG     ;del prescaler distinto al 1:1
BCF STATUS, RP0      ;Banco 0
CLRF TMR0            ;Limpio TMR0 y prescaler antes
BSF STATUS, RP1      ;Banco 1
MOVLW b'xxxx1xxx'    ;Asigno al WDT, no se cambia
MOVWF OPTION_REG     ;valor del prescaler
CLRWDW               ;Limpio WDT y prescaler
MOVLW b'xxxx1xxx'    ;Selección del nuevo valor
MOVWF OPTION_REG     ;del prescaler y asigna al WDT
BCF STATUS, RP0      ;Banco 0
; Líneas 2 y 3 del programa no tienen que ser incluidas si el valor
; del prescaler deseado es distinto a 1:1.
; Si 1:1 es el valor final deseado, entonces un valor de prescaler
; temporal se establece en las líneas 2 y 3 y el valor final
; del prescaler en las líneas 10 y 11.
```

;Cambio de prescaler (WDT -> TIMER0)

```
CLRWDW               ;Limpia WDT y prescaler
BSF STATUS, RP0      ;Banco 1
MOVLW b'xxxx0xxx'    ;Selección TMR0, nuevo valor
                        ;del prescaler y fuente de reloj
MOVWF OPTION_REG ;
BCF STATUS, RP0      ;Banco 0
```

Cálculo de la temporización en TMRO

$$\text{temp TMRO} = [(256 - \text{carga}) \cdot \text{PS} + 2] \cdot T_{\text{instr}}$$

Temporización

Incrementos hasta desbordamiento

PS es el Prescaler, toma el valor 1 si se asigna el prescaler al Watchdog

2 Ciclos de sincronización tras cargar TMRO

$4 / f_{\text{osc}}$

- Se llama **carga** al valor que se asigna al registro TMRO (01h, 101h) al comenzar la temporización.

- El valor del *prescaler* **PS** queda determinado por los tres bits más bajos del registro OPTION_REG (81h, 181h). Si el *prescaler* se le asigna al *watchdog* (bit PSA=1), se tiene **PS=1** para la anterior expresión.

Ejemplo 1 (temp01.asm)

- Se pretende realizar una intermitencia sobre el led controlado desde el pin 3 del PORTB en la placa PICDEM2-PLUS de manera que permanezca 0,5 s encendido y 0,5 s apagado

- La temporización se va a realizar mediante el temporizador TMRO, de manera que, tras realizar una carga del mismo el desbordamiento se produzca a los 500ms. De acuerdo con la anterior fórmula, la temporización máxima que se podría realizar con un oscilador de 4MHz sería:

$$T_{\text{max}} = [(256-0) \cdot 256 + 2] \cdot 4/4\text{MHz} = 65.538 \mu\text{s} = 65,538 \text{ ms} < 500 \text{ ms}$$

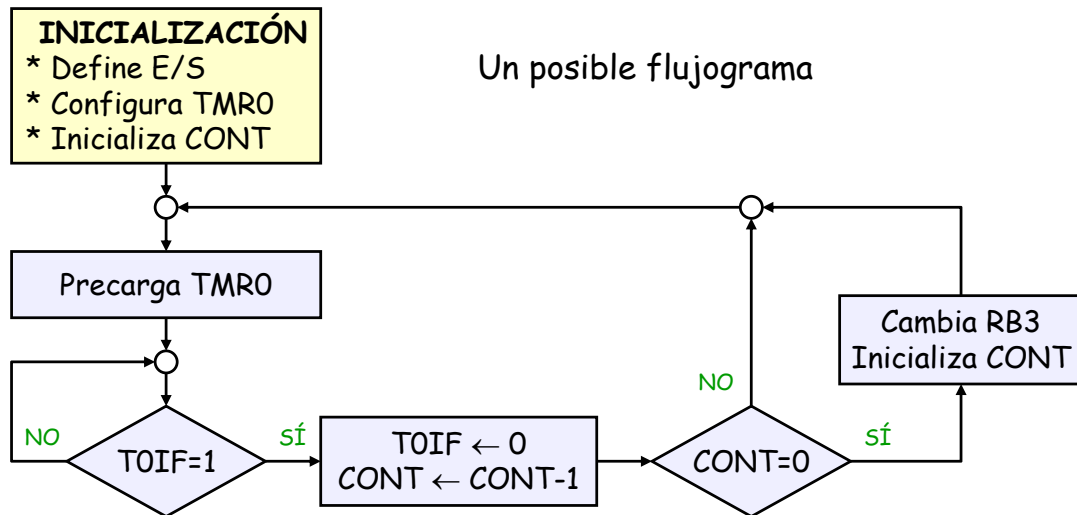
lo que no permite alcanza el tiempo total a temporizar, por tal motivo se emplea un contador que acumule temporizaciones menores hasta alcanzar los 500 ms, por ejemplo se pueden realizar temporizaciones de 50 ms y contabilizar un total de 10. En ese caso, se debe cumplir:

$$50 \text{ ms} = [(256 - \text{Precarga}) \cdot 256 + 2] \cdot 4/4\text{MHz}$$

y despejando en la anterior expresión: Precarga= 60,69 -> **60** (aprox. entera)

Ejemplo 1 (*temp01.asm*)

- Utiliza el temporizador TMRO para hacer que el LED conectado a RB3 parpadee: 500ms encendido y 500ms apagado.



```

***** temp01.asm *****
;
; El LED conectado al bit 3 del Puerto B parpadea de modo que está 500ms encendi-
; do y otros 500ms apagado. Se usa el temporizador TMRO para establecer la tempo-
; rización. (Se considera que el oscilador del PIC es de 4MHz).

; ZONA DE DATOS *****
; Configuración para el grabador
;   _CONFIG _XT_OSC & _WDT_OFF & _PWRTE_ON & _BODEN_ON & _LVP_OFF

LIST      P=16F877 ; Procesador.
INCLUDE <P16F877.INC> ; Definición de los operandos utilizados.

CONT      EQU      0x20 ; Cuenta las veces que se desborda TMRO.

; ZONA DE CÓDIGOS *****
ORG       0 ; El programa comienza en la dirección 0 de
; memoria de programa.
Inicio    bsf       STATUS,RPO ; Pone a 1 el bit 5 de STATUS. Acceso al Banco 1.

          movlw     b'11110111' ; Se configura el bit 3 de PORTB como salida,
          movwf     TRISB ; el resto de bits queda como entradas.

          movlw     0x07 ; Prepara TMRO para contar pulsos de oscilador y
          movwf     OPTION_REG ; le asigna un prescaler de 256.
  
```

```

bcf      STATUS,RPO      ; Pone a 0 el bit 5 de STATUS. Acceso al Banco 0.

clrf     PORTB            ; Inicializa PORTB a 0.
movlw    0x0A             ; Inicializa la variable
movwf    CONT            ; CONT a 10.

```

; Con un oscilador de 4MHz, la máxima temporización que se alcanza con TMRO es de 65,5ms. Para poder llegar a temporizar los 500ms hace falta que TMRO se desborde varias veces. Por ello se preparará TMRO para temporizar 50ms y se esperará que se produzca esta temporización 10 veces.

```

Principal  movlw    d'60'      ; Carga el valor 60 en el registro TMRO (con este
          movwf    TMRO        ; valor se temporizan 50ms).
ESP        btfss    INTCON,TOIF ; Espera que termine la temporización, lo cual se
          goto     ESP          ; detecta cuando el flag TOIF se pone a 1.
          bcf      INTCON,TOIF  ; Baja el flag.
          decfsz   CONT         ; Decrementa CONT. Si sigue siendo distinto de 0,
          goto     Principal    ; lanza una nueva temporización.
          comf     PORTB,F      ; Si CONT=0, cambia el valor de RB3,
          movlw    0x0A         ; y reinicializa la variable CONT
          movwf    CONT;        ; dándole de nuevo el valor CONT=10
          goto     Principal    ; antes de lanzar una nueva temporización.

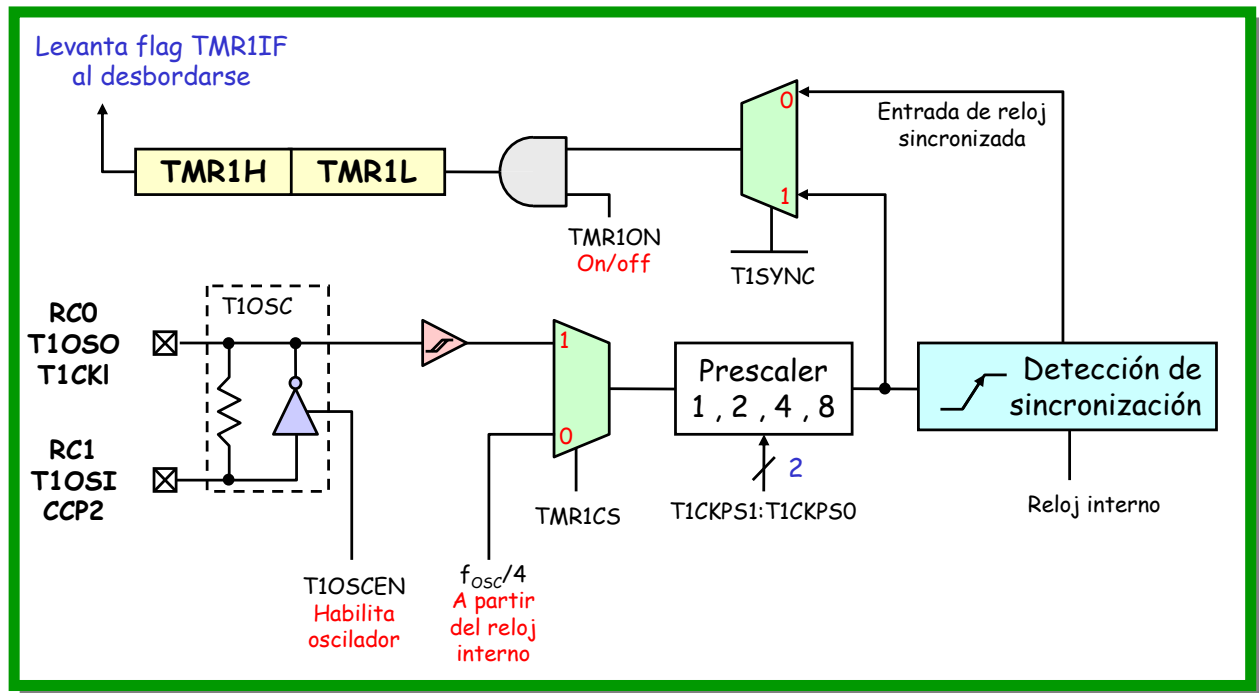
END        ; Fin del programa.

```

Temporizador TMR1: características

- El TMR1 es un **temporizador de 16 bits** basado en un contador ascendente al que se accede a través de **dos registros de 8 bits**: **TMR1H** que almacena los 8 bits que constituyen la parte alta (dirección 0x0Fh) y **TMR1L** que almacena los 8 bits de la parte baja (dirección 0x0Eh). Ambos registros **se pueden leer y escribir** desde el núcleo del microcontrolador.
- TMR1 (TMR1H:TMR1L) puede contar **desde 0x0000 hasta 0xFFFF** (d'65535') y rebosará, iniciando de nuevo la cuenta desde 0x0000; el flag **TMR1IF** (PIR1<0>) **se pone a 1 con ese desbordamiento**.
- La **interrupción del TMR1**, si está habilitada, se producirá en el momento en que se dé el rebosamiento **-overflow-** del TMR1. La interrupción se habilita mediante tres bits: **la máscara particular TMR1IE** (PIE1<0>) y las máscaras **GIE** (global) y **PEIE** (de periféricos) ambas en INTCON que deben estar las dos a 1 (TMR1 tiene una máscara más que el TMRO).
- TMR1 puede **contar** también **flancos externos** entrantes por el pin T1CKI o bien por T1OSI (depende de la activación o no de un oscilador externo)

Diagrama de bloques del TEMPORIZADOR TMR1



T1CON (0x10): Registro de Control del Timer 1

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON
bit 7							bit 0

bit 7-6 No implementados: se leerían como '0'.

bit 5-4 T1CKPS1:T1CKPS0: Bits de selección del prescaler/divisor para la entrada de reloj del Timer1

- 11 = 1:8 valor del prescaler
- 10 = 1:4 valor del prescaler
- 01 = 1:2 valor del prescaler
- 00 = 1:1 valor del prescaler

bit 3 T1OSCEN: Bit de habilitación (enable) del oscilador interno del TMR1

- 1 = Oscilador habilitado
- 0 = Oscilador apagado

bit 2 T1SYNC: Bit de control de la sincronización del reloj externo del Timer1

Cuando TMR1CS=1:

- 1 = No sincroniza la entrada de reloj externo
- 0 = Sincroniza la entrada de reloj externo

Cuando TMR1CS = 0:

Este bit se ignora, pues TIMER1 usa el reloj interno que ya está sincronizado.

bit 1 TMR1CS: Bit de selección de reloj para el TIMER 1

- 1 = Reloj externo del pin RC0/T1OSO/T1CKI (flancos de subida)
- 0 = Reloj interno (FOSC/4)

bit 0 TMR1ON: Bit para arranque/paro del Timer1.

- 1 = Timer1 cuenta pulsos
- 0 = Timer1 parado

Cálculo de la temporización con TMR1

$$\text{temp TMR1} = [(65536 - \text{precarga}) \cdot \text{PS}] \cdot T_{\text{instr}}$$

Temporización

Incrementos hasta desbordamiento

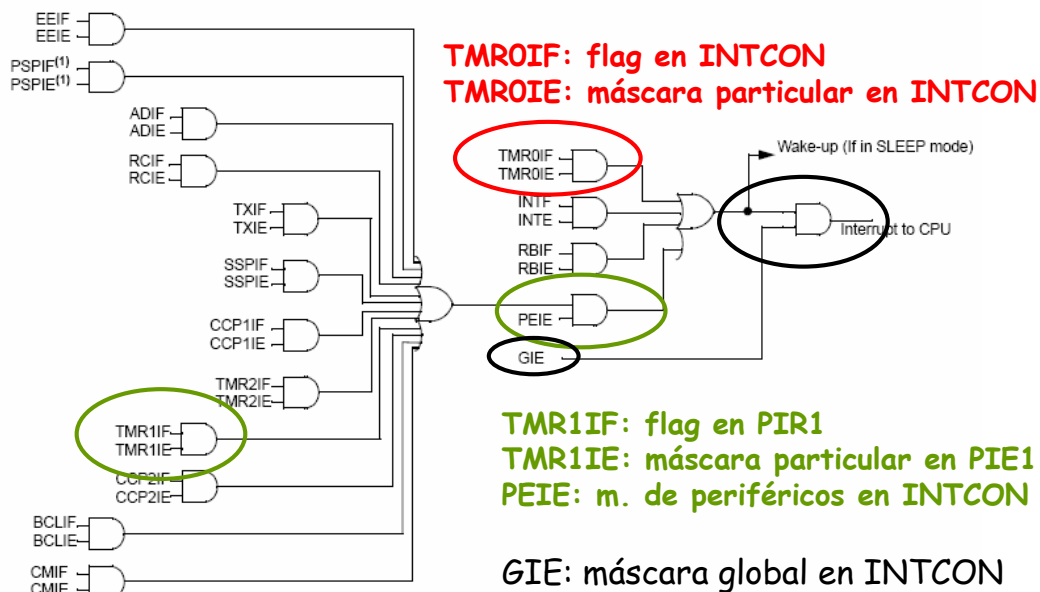
PS es el Prescaler, toma el valor 8 como máximo

4 / fosc

• Se llama **precarga** al valor que se asigna al registro TMR1 en su conjunto al comenzar la temporización, se descompondrá en un número binario de 16 bits los 8 bits altos se cargarán en TMR1H y los 8 bits bajos en TMR1L.

• El valor del **prescaler PS** queda determinado por dos bits del registro T1CON (10h): T1CKPS1:T1CKPS0

Lógica de interrupciones para TMR1 y para TMR0



0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE

Temporizador TMR1

- El temporizador TMR1 tiene 2 modos de funcionamiento:
 - Como **temporizador**
 - Como **contador de flancos externos**
 El modo de funcionamiento viene determinado por el bit "clock select" **TMR1CS** (T1CON<1>).
- En modo **temporizador**, el **TMR1 se incrementa cada ciclo de instrucción** si no se use el prescaler ó divisor previo o bien cada **varios ciclos de instrucción** (dependiendo del valor del prescaler).
- En modo **contador**, el TMR1 se incrementa en cada **flanco ascendente** de una señal de reloj externa - también si no se usa el prescaler-. Este reloj puede provenir de una **señal de reloj externa** que entra al microcontrolador por la patilla RC0/T1OSO/T1CKI ó puede provenir de un **oscilador propio** (cristal de cuarzo o resonador cerámico) situado entre las patillas RC0/T1OSO/T1CKI y RC0/T1OSO/T1CKI. La frecuencia de este oscilador suele ser totalmente distinta de la del oscilador utilizado para el ciclo de instrucción del micro, para hacer temporizaciones más largas que las que permite el oscilador del microcontrolador.
- A diferencia del TMR0 cuya cuenta no puede detenerse, el TMR1 tiene la posibilidad de **activar/parar** la cuenta mediante el **bit TMR1ON** (T1CON<0>).
- El TMR1 también tiene una **entrada de RESET** que lo pondría a 0000h en el momento en que se activa. Esta entrada de RESET está **controlada por los dos módulos CCP** disponibles (véanse las características de los módulos CCP).

TMR1: MODO TEMPORIZADOR

El modo **temporizador** se selecciona al poner a 0 el bit TMR1CS (T1CON<1>). En este modo la entrada de reloj es de **frecuencia fosc/4**. Esta frecuencia puede utilizarse directamente para la cuenta de TMR1 (si T1CKPS1:T1CKPS0=b'00') o **puede dividirse previamente en el "prescaler"** si estos bits tienen un valor distinto a b'00'. El **bit de control de sincronismo** (T1SYNC) no tiene efecto en este modo de funcionamiento pues el TMR1 se incrementará ya de manera síncrona siempre en la fase Q3 del ciclo de instrucción que le corresponda.

TMR1: MODO CONTADOR

- El modo **contador** se selecciona al poner a 1 el bit TMR1CS (T1CON<1>). En este modo la entrada de reloj puede **provenir de un sistema externo** y, por tanto, entrar al TMR1 a través de la patilla RC0/T1OSO/T1CKI ó de un **oscilador** colocado entre las patillas RC0/T1OSO/T1CKI y RC1/T1OSI/CCP2. En este último caso el bit T1OSCEN (T1CON<3>) debe estar a 1.
- En modo **contador**, el **incremento se produce siempre con el flanco de subida** de la señal de reloj (no hay opción de seleccionar el flanco como en TMR0).
- TMR1 en modo contador tiene **dos submodos posibles: síncrono o asíncrono**. Depende del valor del bit T1SYNC (T1CON<2>).



TMR1: MODO CONTADOR SÍNCRONO (TMR1CS = 1; T1SYNC=0)

- El modo **contador síncrono** incrementa el temporizador (o bien el prescaler) en cada flanco de subida de la entrada de reloj de la patilla RC1/T1OSI/CCP2 (si T1OSCEN = 1) ó de la patilla RC0/T1OSO/T1CKI (si T1OSCEN = 0).
- Al estar el bit T1SYNC a 0, el **reloj externo** sufre un proceso de sincronización con el reloj de fase interno (Q1-Q2-Q3-Q4) después del prescaler. El **prescaler es un contador asíncrono**, es decir, se incrementa con cada flanco de reloj de entrada hasta su cuenta (definida por los bits T1CKPS1:T1CKPS0) y luego vuelve a cero.
- En modo dormido (**SLEEP**), el TMR1 no se incrementa si está programado en **modo contador sincronizado** aunque exista reloj externo, no podría por tanto "despertar" al microcontrolador en este modo. Ello se debe a que no **hay reloj de fase para la sincronización** y por tanto el circuito de sincronización está apagado. El "prescaler", no obstante si que se incrementaría.



TMR1: MODO CONTADOR SÍNCRONO (TMR1CS = 1; T1SYNC=0)

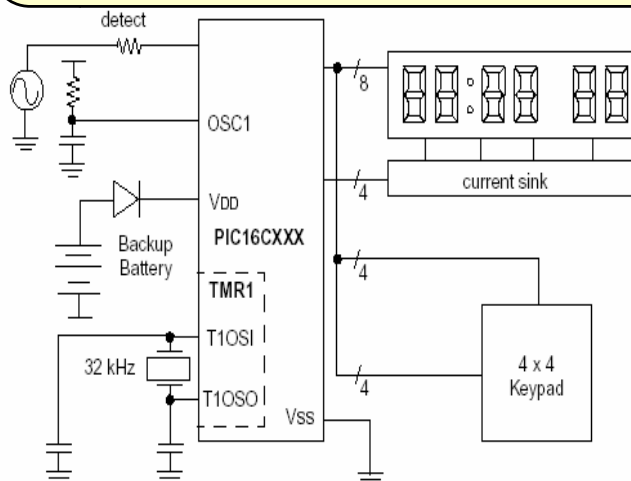
- En este modo de funcionamiento el reloj externo debe cumplir ciertos **requisitos** debido al proceso de **sincronización** que sufre con el reloj de fase interno. También **existirá un retardo** entre el **flanco** y el **incremento real** del TMR1.
- Cuando el prescaler es 1:1, la entrada de reloj es la misma señal que la salida del prescaler. El proceso de sincronización de T1CKI se realiza **muestreando la salida del prescaler cada 2 T_{osc}** (al final de las fases Q2 y Q4) y, por tanto, el requisito para la señal de reloj de T1CKI es que **esté al menos durante 2T_{osc} en estado alto** y otros **2T_{osc} en estado bajo** (véanse parámetros del TMR1).
- Cuando se usa un prescaler distinto al 1:1, la frecuencia de la **señal de reloj externa se divide con el prescaler**. Para que el reloj externo cumpla los requisitos para el proceso de sincronizado, la señal que entra por T1CKI debe tener un periodo de al menos 4T_{osc}/Prescaler (+ un pequeño retardo RC). El tiempo que debe estar en **estado alto o bajo** la señal de reloj debe cumplir al menos los **requisitos de mínimo ancho de pulso** (véanse parámetros del TMR1 a continuación).

Parámetros del TMR1

Param No.	Symbol	Characteristic	Min	Typ†	Max	Units	Conditions
45*	Tt1H	T1CKI High Time	Synchronous, Prescaler = 1	0.5T _{CY} + 20	—	—	ns Must also meet parameter 47
			Synchronous, Prescaler = 2,4,8	Standard(F)	15	—	
			Extended(LF)	25	—	—	
			Asynchronous	Standard(F)	30	—	
			Extended(LF)	50	—	—	
46*	Tt1L	T1CKI Low Time	Synchronous, Prescaler = 1	0.5T _{CY} + 20	—	—	ns Must also meet parameter 47
			Synchronous, Prescaler = 2,4,8	Standard(F)	15	—	
			Extended(LF)	25	—	—	
			Asynchronous	Standard(F)	30	—	
			Extended(LF)	50	—	—	
47*	Tt1P	T1CKI input period	Synchronous	Standard(F)	Greater of: 30 OR $\frac{T_{CY} + 40}{N}$	—	ns N = prescale value (1, 2, 4, 8)
				Extended(LF)	Greater of: 50 OR $\frac{T_{CY} + 40}{N}$	—	ns N = prescale value (1, 2, 4, 8)
			Asynchronous	Standard(F)	60	—	ns
				Extended(LF)	100	—	ns
	Ft1	Timer1 oscillator input frequency range (oscillator enabled by setting bit T1OSCEN)	DC	—	200	kHz	
48	TCKEZtmr1	Delay from external clock edge to timer increment	2T _{OSC}	—	7T _{OSC}	—	

TMR1: MODO OSCILADOR (idem a contador síncrono con T1OSCEN = 1)

Los PIC16F87x tienen internamente la **circuitería necesaria** para generar una señal de reloj basándose en un cristal de cuarzo ó un resonador cerámico **conectado entre las patillas RC0/T1OSO/T1CKI y RC1/T1OSI/CCP2** e independiente del oscilador del microcontrolador. El **circuito del oscilador se debe activar** mediante la puesta a 1 del bit **T1OSCEN** (si este bit se activa, las líneas T1OSI y T1OSO pasan a ser entradas independientemente de la carga del TRISC)



Este oscilador puede llegar a ser de hasta 200 KHz, pero la frecuencia más habitual es de 32,768 KHz que es una frecuencia que se ajusta muy bien para aplicaciones de tiempo real ya que pueden extraerse muy fácil de este reloj unidades de segundo.

Prescale	Frequency (kHz)		
	32.768	100	200
1	2	0.655	0.327
2	4	1.31	0.655
4	8	2.62	1.31
8	16	5.24	2.62

Overflow times in seconds.

TMR1H Load Value	Overflow Time (@ 32.768 kHz)
80h	1 Second
C0h	0.5 Second
E0h	0.25 Second
F0h	0.125 Second

— Ejemplo típico para un reloj

TMR1: MODO CONTADOR ASÍNCRONO (TMR1CS = 1; T1SYNC=1)

- A diferencia del modo síncrono, el reloj externo (proveniente de cualquiera de las patillas) **no está sincronizado** con el reloj de fase de la instrucción y por tanto el TMR1 se incrementa **en el momento en que se produzca el flanco de subida en la señal** de reloj (si no se usa prescaler) o en el momento que se produce el desbordamiento del prescaler (si se usa prescaler).
- En modo contador asíncrono, el TMR1 **sigue contando pulsos si se encuentra en modo dormido**. Al desbordarse el contador, se **podría provocar** (si las máscaras correspondientes están activas) **que el microcontrolador despierte si es que se encontraba dormido**.
- En el modo contador asíncrono, el TMR1 **no se puede usar como base de tiempos** para operaciones de **captura o comparación** (se explicará al hablar de módulos CCP).
- En el modo contador asíncrono, como el incremento del TMR1 **se puede producir en cualquier momento del ciclo de instrucción**, a la hora de leer ó escribir en el TMR1 se deben tomar ciertas precauciones, según se expone a continuación.

TMR1: Escritura del valor de 16 bits

- Para cargar un valor dado en el contador TMR1, se **recomienda parar la cuenta del TMR1** mientras se escriban los valores en los registros TMR1H y TMR1L. Podría producirse la **carga de un valor no predecible** en alguno de los registros si a la vez que se intenta cargar un nuevo valor por programa, se produce un flanco en la señal de reloj externa que también estará modificando (incrementando) el valor de esos registros.
- Si **no fuera posible la detención**, existen otras posibles soluciones según se ilustra en el siguiente ejemplo: se quiere cargar HI_BYTE en TMR1H y LO_BYTE en TMR1L

```

;Secuencia para la escritura del TMR1
;Todas las interrupciones deshabilitadas para que no haya incrementos
CLRF    TMR1L    ; Limpia byte bajo, asegurando que no
;se desborde TMR1L ya que provocaría el incremento de TMR1H
MOVLW   HI_BYTE ; Valor a cargar en TMR1H a W
MOVWF   TMR1H    ; Escribo el byte alto ahora
MOVLW   LO_BYTE ; Valor a cargar en TMR1L a W
MOVWF   TMR1L    ; Escribo ahora el byte bajo
; Vuelvo a habilitar las interrupciones (si se requiere)
...
```

TMR1: Lectura de TMR1H y TMR1L

La lectura de los 16 bits del temporizador TMR1 no puede hacerse de **manera simultánea** sino que debe realizarse de manera individual para cada uno de los 2 paquetes de 8 bits. Podría darse el caso de que el TMR1 rebosara en medio de las dos lecturas y esto originaría una lectura incorrecta.

Ejemplo: leer y guardar en TMPH:TMPL

; Secuencia para la lectura correcta del TMR1

; Todas las interrupciones deshabilitadas

MOVF TMR1H, W ; Leo byte alto de TMR1 -1ª lectura

MOVWF TMPH ;

MOVF TMR1L, W ; Leo byte bajo de TMR1

MOVWF TMPL ;

MOVF TMR1H, W ; 2ª lectura del byte alto de TMR1

SUBWF TMPH, W ; Resta la 1ª y la 2ª lectura

BTFSC STATUS, Z ; si son iguales, la resta dio 0

GOTO CONTINUE ; Continuo el programa con la lectura correcta

;

; Si llegamos aquí TMR1L se desbordó en medio de la 1ª lectura del byte alto

; y del byte bajo. Debo volver a leerlos para obtener un valor correcto

;

MOVF TMR1H, W ; Leo de nuevo el byte alto

MOVWF TMPH ;

MOVF TMR1L, W ; Leo de nuevo el byte bajo

MOVWF TMPL ;

; Aquí habilitaría de nuevo las interrupciones (si se requiere)

CONTINUE ; Continuamos con el programa

.....

TMR1	Sequence 1		Sequence 2	
	Action	TMPH:TMPL	Action	TMPH:TMPL
04FFh	READ TMR1L	xxxxh	READ TMR1H	xxxxh
0500h	Store in TMPL	xxFFh	Store in TMPH	04xxh
0501h	READ TMR1H	xxFFh	READ TMR1L	04xxh
0502h	Store in TMPH	05FFh	Store in TMPL	0401h

Procedimiento
correcto de
lectura segura

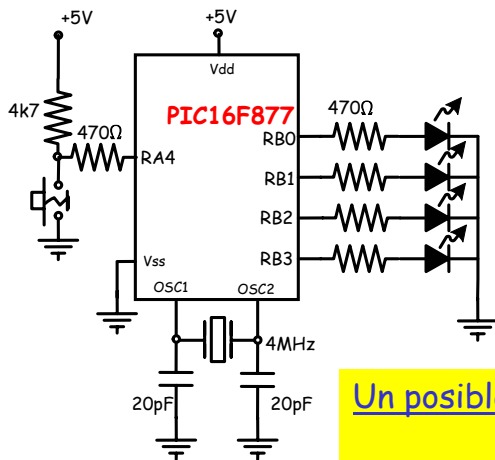
Registros asociados al funcionamiento de TMR1 como Temporizador/Contador

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
0Eh	TMR1L	Holding Register for the Least Significant Byte of the 16-bit TMR1 Register								xxxx xxxx	uuuu uuuu
0Fh	TMR1H	Holding Register for the Most Significant Byte of the 16-bit TMR1 Register								xxxx xxxx	uuuu uuuu
10h	T1CON	—	—	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON	--00 0000	--uu uuuu

Observaciones:

- TMR1H y el TMR1L **no se resetean** a 00h en un POR ó en cualquier otro RESET
- T1CON se pone a 0x00 en un POR ó en un Brown-out Reset colocando por tanto al TMR1 en estado de **apagado** y con **prescaler 1:1**. En el resto de los RESET el registro no se ve modificado.

ENUNCIADO: Se desea realizar un programa que utiliza los siguientes elementos de la placa PICDEM2-PLUS: los led conectados al PORTB y el pulsador conectado al pin 4 del PORTA y desarrolla la siguiente acción:



Al principio, aparecen todos los led encendidos y así continúan hasta que se actúa sobre el pulsador, en ese momento se apagan todos los led y se produce una intermitencia en el led conectado a RB3 de manera que está 0,4 s encendido y 0,4 s apagado y esto durante un total de 5 s, al cabo de los cuales se retorna al estado inicial.

Un posible planteamiento:

- Se temporizan los 0,4 s con TMR0
- Se temporizan los 5 s con TMR1
- La vuelta al estado inicial se consigue dejando que el Watchdog desborde y resetee al microcontrolador

Cálculos:

TMR0, temporización máxima = $[(256-0)*256+2] * 4/4\text{MHz} = 65,538 \text{ ms}$

Como no podemos alcanzar los 400 ms usaremos temporizaciones de 40 ms y un contador de interrupciones en CONTO que se cargará con 10 y se irá decrementando hasta completar los 400 ms = $10 * 40 \text{ ms}$. Al usar TMR0 el prescaler, WDT no puede utilizarlo y desbordará al cabo de 18 ms si antes no se le pone a cero con CLRWDT

$$40 \text{ ms} = [(256-\text{precarga}) * 256 + 2] * 1 \mu\text{s} \quad \Rightarrow \quad \text{precarga TMR0} = 100$$

TMR1, temporización máxima = $[(65536-0)*8] * 4/4\text{MHz} = 524,288 \text{ ms}$

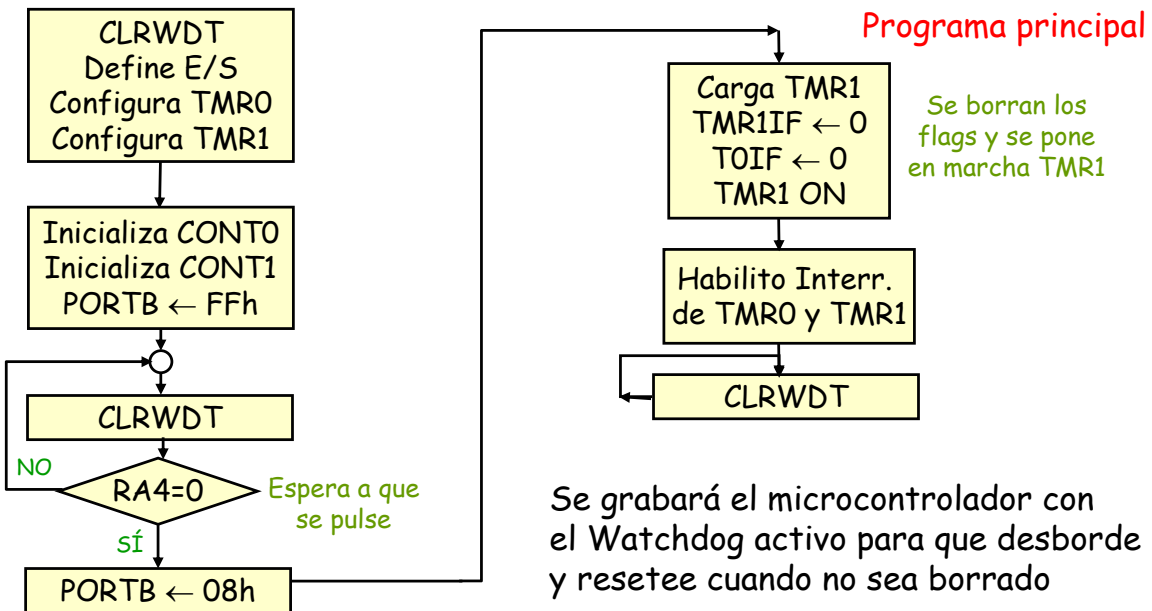
Tampoco podemos alcanzar los 5 s y usaremos la misma estrategia, un contador de temporizaciones de 500 ms por ejemplo y un contador de interrupciones en CONT1 que se cargará con 10 y se irá decrementando hasta completar los 5 s = $10 * 500 \text{ ms}$

$$500 \text{ ms} = [(65536-\text{precarga}) * 8] * 1 \mu\text{s}$$

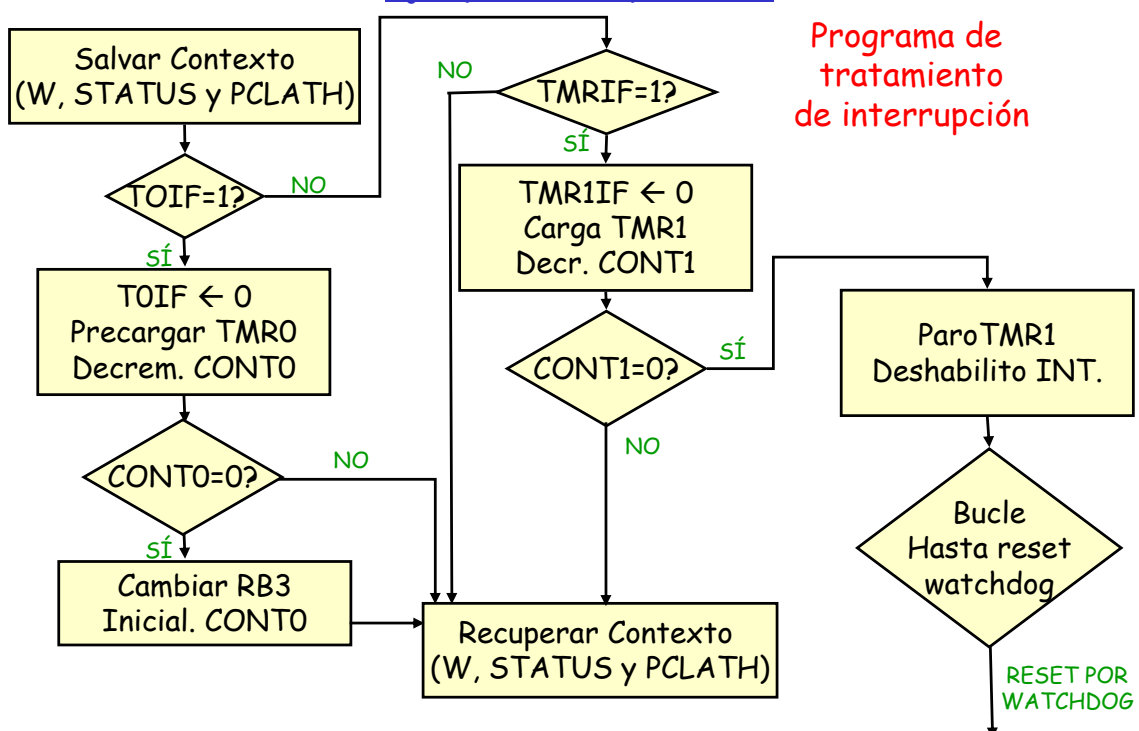
$$\text{precarga TMR1} = 3036 = 0x0BDC$$

Ejemplo 2 (temp04.asm)

• Se utiliza el temporizador TMRO para hacer que el LED conectado a RB3 parpadee: 400ms encendido y 400ms apagado. Este parpadeo sólo se produce durante 5s (temporizados con TMR1). La vuelta al estado inicial se provoca con un reset por desbordamiento del watchdog. A los 5 segundos deben añadirse los 18 mseg. típicos de desbordamiento del watchdog sin prescaler.



Ejemplo 2 (temp04.asm)





```

***** temp04.asm *****
; Inicialmente todos los LEDs del Puerto B aparecen encendidos. A partir de
; esta situación, cuando se accione el pulsador conectado a RA4, el LED conectado
; al bit 3 del Puerto B parpadea de modo que está 400ms encendido y otros 400ms
; apagado. Se usa el temporizador TMRO para establecer esta temporización.
; Al mismo tiempo, el temporizador TMR1 lleva a cabo una temporización de cinco
; segundos, transcurridos los cuales cesa el parpadeo y se recupera la situación
; inicial (todos los LEDs encendidos y espera por pulsación de RA4) debido a
; que el micro se resetea por un watchdog.
; Se considera que el oscilador del PIC es de 4MHz.

; ZONA DE DATOS *****
; Configuración para el grabador
__CONFIG __XT_OSC & __WDT_ON & __PWRTE_ON & __BODEN_ON & __LVP_OFF

LIST      P=16F877 ; Procesador.
INCLUDE <P16F877.INC> ; Definición de los operandos utilizados.

W_TEMP    EQU    0x20    ; Guarda valor de W durante interrupción.
STATUS_TEMP EQU    0x21    ; Guarda valor de STATUS durante interrupción.
PCLATH_TEMP EQU    0x22    ; Guarda valor de PCLATH durante interrupción.
CONTO      EQU    0x23    ; Cuenta las veces que desborda TMRO.
CONT1      EQU    0x24    ; Cuenta las veces que desborda TMR1.

```



```

; ZONA DE CÓDIGOS *****
ORG      0    ; Comienzo en la dirección 0 de memoria de programa.
goto     Inicio ; Vector de RESET.
ORG      4
goto     RTI    ; Vector de tratamiento de interrupción.

; Programa Principal -----
Inicio   clrwdt    ; Reset del Watchdog
        bsf       STATUS,RPO ; Paso al Banco 1.
        clrf      TRISB    ; Se configuran los bits de PORTB como salidas.
        movlw     b'00000111' ; Prepara TMRO para contar pulsos de oscilador y
        movwf     OPTION_REG ; le asigna un prescaler de 256.
        bsf       PIE1,TMR1IE ; Habilita la interrupción de TMR1.
        bcf       STATUS,RPO ; Vuelta al Banco 0.
        movlw     b'00110000' ; Prepara TMR1 para contar pulsos de oscilador y
        movwf     T1CON    ; le asigna un prescaler de 8. Se deja apagado al inicio.
        movlw     d'10'    ; Inicializa la variable
        movwf     CONTO    ; CONTO a 10.
        movwf     CONT1    ; y también CONT1 a 10.
        movlw     0xFF     ; Inicializa PORTB a FFh para que aparezcan todos
        movwf     PORTB    ; sus LEDs encendidos.

PULSA    clrwdt    ; para evitar el desbordamiento del watchdog
        btfsc     PORTA,4  ; Si no se acciona el pulsador (RA4=1),
        goto     PULSA    ; se sigue esperando.
        movlw     0x08     ; En caso contrario, se apagan todos los LEDs
        movwf     PORTB    ; menos el de RB3.

```




```

; Se prepara TMRO para temporizar 40ms y se esperará a que se produzca esta temporización
; 10 veces para completar los 0,4 s.
    movlw    d'100'          ; Carga el valor 100 en el registro TMRO (con este
    movwf    TMRO            ; valor se temporizan 40ms).
    bcf      INTCON,TOIF     ; Ponemos a cero el flag de desbordamiento de TMRO
; Para poder llegar a temporizar los 5 s hace falta que TMR1 desborde varias veces. Por ello se
; preparará TMR1 para temporizar 500ms y se esperará esta temporización 10 veces.
    movlw    0x0B           ; Se carga TMR1 con el valor 3036 (0x0BDC), que
    movwf    TMR1H          ; da lugar a una temporización de 500ms antes
    movlw    0xDC           ; de que se desborde (según la configuración
    movwf    TMR1L          ; establecida inicialmente).
    bsf      T1CON,TMR1ON   ; Se pone en marcha TMR1
    bcf      PIR1,TMR1IF    ; Se asegura de que está bajado el flag de TMR1.
    movlw    b'11100000'    ; Baja flag de TMRO, se habilita su máscara
    movwf    INTCON         ; y la máscara global de interrupciones.
Bucle clrwdt                ; Se limpia el watchdog para evitar desbordamientos,
    goto     Bucla          ; de este bucle sólo se sale por interrupción
                                ; de TMRO ó de TMR1

; Subprograma de Tratamiento de Interrupción-----
RTI   movwf    W_TEMP       ; Salvamos el registro W.
      swapf    STATUS,W     ; Guardamos el registro STATUS "girado" en W.
      bcf      STATUS,RPO   ; Aseguramos el paso al Banco 0.
      bcf      STATUS,RP1
      movwf    STATUS_TEMP   ; Guardamos STATUS (girado) en el Banco 0.
      movf     PCLATH,W      ; Salvamos también PCLATH
      movwf    PCLATH_TEMP

```



```

    btfsc    INTCON,TOIF    ; Esta sección se encarga de identificar el
    goto     RTI_TMRO       ; origen de la interrupción. El orden en que
    btfsc    PIR1,TMR1IF    ; se comprueba es: 1) Desbordamiento
    goto     RTI_TMR1       ; de TMRO y 2) Desbordamiento de TMR1.
    goto     SAL_RTI        ; si ninguno de los flags está a 1, salimos directamente

RTI_TMRO ; Tratamiento de la interrupción provocada por desbordamiento de TMRO
    bcf      INTCON,TOIF    ; Baja el flag de TMRO.
    movlw    d'100'         ; Vuelve a cargar TMRO con el valor necesario
    movwf    TMRO           ; para llevar a cabo una nueva temporización deseada.
    decfsz   CONTO          ; Decrementa CONTO. Si su valor no pasa a ser 0,
    goto     SAL_RTI        ; sale de la interrupción (no pasaron 0,4 s).
    movlw    b'00001000'    ; Si no, es que pasaron 0,4 s y hay que
    xorwf    PORTB,F        ; cambiar el valor a sacar por RB3,
    movlw    d'10'          ; recargamos el contador de interrupciones de TMRO
    movwf    CONTO          ; CONTO a diez
    goto     SAL_RTI        ; y se sale de la interrupción.

RTI_TMR1 ; Tratamiento de la interrupción provocada por desbordamiento de TMR1
    bcf      PIR1,TMR1IF    ; Baja el flag de TMR1.
    movlw    0x0B           ; Vuelve a cargar TMR1 con el valor necesario
    movwf    TMR1H          ; para llevar a cabo la temporización deseada.
    movlw    0xDC           ; no hay peligro de que TMR1L desborde mientras
    movwf    TMR1L          ; realizamos esta operación y cambie TMR1H
    decfsz   CONT1          ; Decrementa CONT1. Si su valor no pasa a ser 0,
    goto     SAL_RTI        ; se sale de la interrupción directamente.
    bcf      T1CON,TMR1ON   ; Si no, se para el temporizador TMR1 antes
    bcf      INTCON,GIE     ; y se deshabilitan las interrupciones

```



; Como ya transcurrieron los 5 segundos, nos quedamos en este bucle infinito sin resetear el
; Watchdog hasta que desborde y reinicie el microcontrolador

bucleinf goto bucleinf ;tras 18 mseg (valor típico)
;el micro se reseteará por desbordamiento del Watchdog

; Recuperación del contexto almacenado al entrar en el programa de tratamiento

SAL_RTI

```
movf    PCLATH_TEMP,W    ;Recuperamos PCLATH.
movwf   PCLATH
swapf   STATUS_TEMP,W    ;Recuperamos el registro STATUS con un SWAPF.
movwf   STATUS
swapf   W_TEMP,F         ;Recuperamos también el W con dos SWAPF.
swapf   W_TEMP,W
retfie                                     ;Retorno al programa principal.

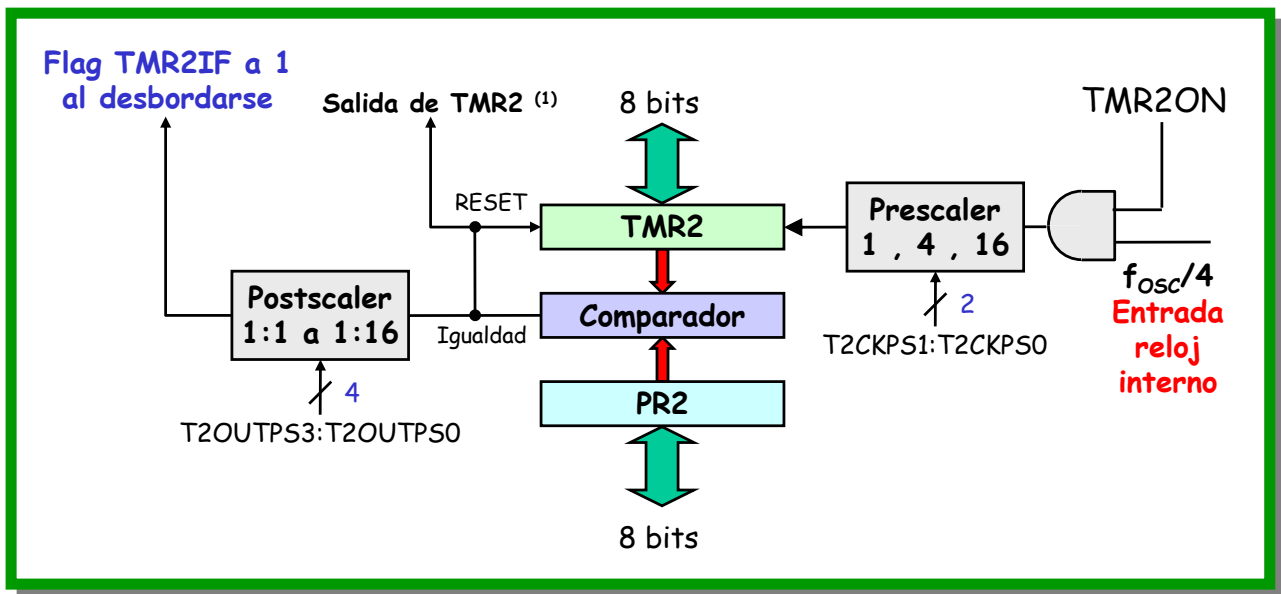
END                                       ; Fin del programa.
```



Temporizador TMR2: características

- TMR2 es un temporizador de **8 bits** con un **prescaler** (divisor de frecuencia previo), un **registro de periodo** (PR2) que marca el valor máximo que puede alcanzar la cuenta de TMR2 y un **postscaler** (contador de coincidencias entre TMR2 y PR2)
- El registro **TMR2 se puede leer y escribir** desde el núcleo del microcontrolador. TMR2 puede trabajar como **temporizador** pero **no como contador de flancos externos**
- El contador TMR2 puede contar desde **0x00 hasta el valor cargado en PR2**, en el ciclo siguiente al de esa coincidencia, el contador vuelve a cero
- El TMR2 también se puede utilizar para generar una **señal de reloj** para **transferencias serie síncronas** mediante el puerto serie síncrono (véase módulo SSP)
- El TMR2 se emplea además como **base de tiempos** para **los módulos CCP** cuando se configuran en modo PWM (véanse módulos CCP).

Diagrama de bloques del TEMPORIZADOR TMR2



⁽¹⁾ La salida de TMR2 puede ser usada por el módulo SSP como reloj para transferencia serie síncrona

T2CON: REGISTRO DE CONTROL DEL TIMER 2

dirección 0x12 de RAM

U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0
bit 7						bit 0	

bit 7 No implementado: Se lee como 0

bit 6:3 TOUTPS3:TOUTPS0: bits de selección del postscaler del Timer2

0000= 1:1 Postscale

0001= 1:2 Postscale

.

.

.

1111= 1:16 Postscale

bit 2 TMR2ON: Bit de paro/arranque del TMR2

1 = Timer2 on

0 = Timer2 off

bit 1:0 T2CKPS1:T2CKPS0: Bits de selección de prescaler del Timer2

00= Prescaler is 1

01= Prescaler is 4

1x= Prescaler is 16

Cálculo de la temporización con TMR2

$$\text{temp TMR2} = [\text{Prescaler} \cdot (\text{PR2}+1) \cdot \text{Postscaler}] \cdot T_{\text{instr}}$$

Temporización

El Prescaler es un divisor de frecuencia previo al TMR2

TMR2 presenta PR2+1 combinaciones diferentes en su salida antes de volver a cero

El Postscaler cuenta el número de coincidencias entre TMR2 y PR2 antes de poner TMR2IF=1

4/fosc

- En este caso, se pueden elegir **tres valores distintos** (Prescaler, PR2 y Postscaler) a la hora de ajustar el valor de **temporización**. Dispondremos de "dos grados de libertad", teniendo en cuenta que todos son valores enteros.
- En la mayoría de los casos se podrá hacer una **mejor aproximación** al valor deseado que en el caso de utilizar TMRO (el Postscaler puede ser 1,2,3,4..16)

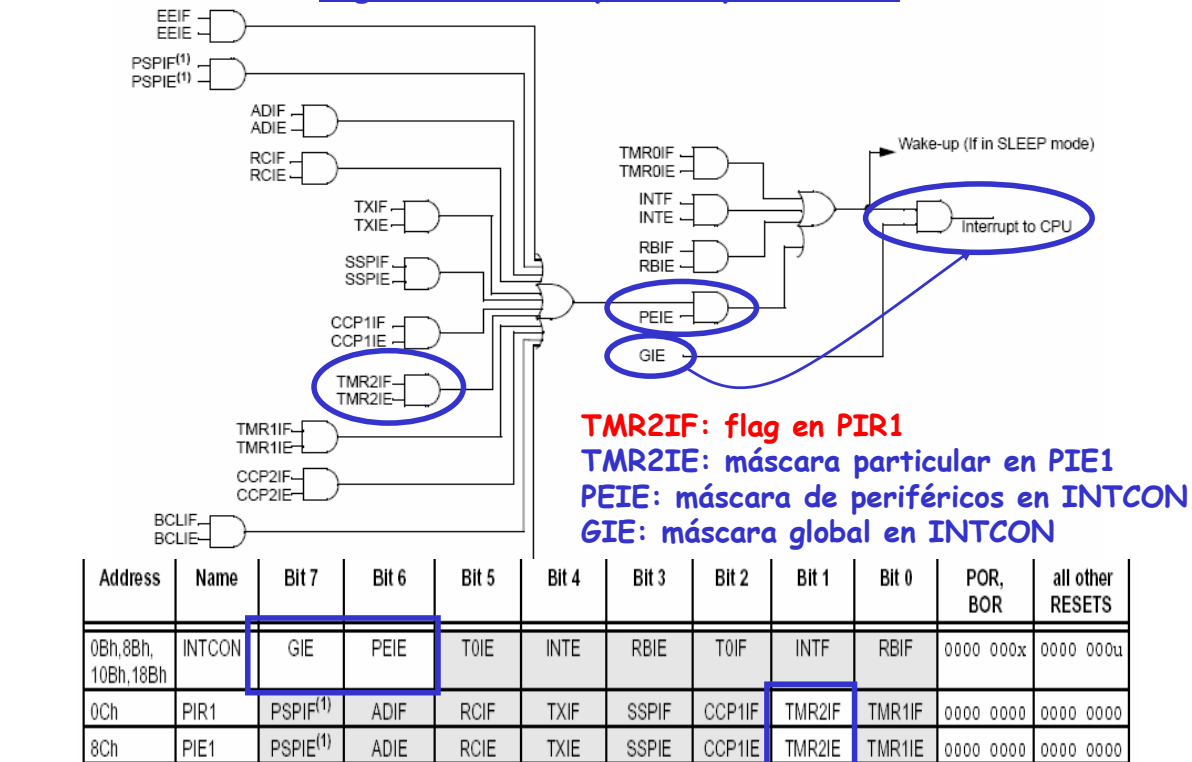
Operación con Temporizador TMR2

- En el caso de TMRO y TMR1 era necesario **precargar un valor de partida** en la cuenta y el **flag** correspondiente se activaba **al desbordar** el contador, esto obligaba a volver a **precargar de nuevo** el contador para iniciar una nueva temporización. En el caso de TMR2, la **cuenta va desde 0x00 hasta el valor de PR2** y en el ciclo siguiente el contador vuelve a cero por hardware, con lo cual **no es necesario precargar por software ningún valor periódicamente** y las temporizaciones tendrán una duración fija mientras no se modifique el registro de periodo (PR2). No obstante, también sería posible precargar un valor en TMR2 aunque no será lo habitual más que en el principio de todo el proceso.
- El flag **TMR2IF** se pondrá a 1 cuando produzca un **número de coincidencias entre TMR2 y PR2** igual al valor establecido para el **postscaler**
- Si usamos el **prescaler** y el **postscaler** con su valor máximo, el tiempo de **rebosamiento máximo del postscaler (overflow)** sería:

$$(4/\text{fosc}) \cdot (\text{Prescaler máximo}) \cdot 256 \cdot (\text{Postscaler máximo}) = (4/\text{fosc}) \cdot 2^{16}$$

que supondría un valor de **temporización máximo** igual que el que se puede conseguir con el temporizador **TMRO** pero **inferior al máximo alcanzable con TMR1**

Lógica de interrupciones para TMR2



Resets del Temporizador TMR2

- El registro **TMR2** se pone a cero en todos los reset del microcontrolador y ante estos el registro **PR2** se carga con 0xFF (d'255').
- El **TMR2** se puede parar (deshabilitar el incremento) si se pone a cero el bit **TMR2ON** (T2CON<2>). Hay pues bit de parada y puesta en marcha
- La salida del comparador que se activa cuando TMR2 iguala al valor del PR2 se utiliza como entrada a dos bloques:
 - ❖ Al postscaler
 - ❖ A la entrada de reloj del módulo SSP en transferencia serie síncrona
- Para la selección del postscaler disponemos de 4 bits TOUTPS3:TOUTPS0 (T2CON<6:3>) que permiten elegir desde 1:1 hasta 1:16 (ambos inclusive).
- El prescaler y el postscaler se ponen a cero cada vez que se produce una escritura del registro **TMR2** ó una escritura del registro **T2CON** ó con cualquier tipo de RESET en el dispositivo. Hay que tener en cuenta, no obstante que cuando se escribe en el registro **T2CON**, el registro **TMR2** no se pone a cero.

REGISTROS ASOCIADOS AL TIMER 2

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh,8Bh, 10Bh,18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
11h	TMR2	Timer2 Module's Register								0000 0000	0000 0000
12h	T2CON	—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0	-000 0000	-000 0000
92h	PR2	Timer2 Period Register								1111 1111	1111 1111

Durante el modo dormido o de bajo consumo (SLEEP), el TMR2 **no se incrementa ya que no hay oscilador interno**, pero el **prescaler** y **TMR2** retienen sus **últimos valores** y el módulo está preparado para **reanudar la cuenta** cuando el **dispositivo despierte** del modo SLEEP.